

SV9 - Digital Verification using SV & UVM

=====Assignment(1)=====

Name
Ibrahim Mohamed Mohamed Hanafy

Design 1: Adder

-Adder Design Code:

```
1  module adder (  
2      input clk,  
3      input reset,  
4      input signed [3:0] A, // Input data A in 2'  
5      input signed [3:0] B, // Input data B in 2'  
6  
7      output reg signed [4:0] C // Adder output  
8  );  
9  
10 // Register output C  
11 always @(posedge clk or posedge reset) begin //active high  
12     if (reset)  
13         C <= 5'b0;  
14     else  
15         C <= A + B;  
16 end  
17  
18 endmodule
```

-Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functionality Check
Adder	-When the reset is asserted HIGH, the output value should be all 0's -Handle output range: [-16,14] -100% Coverage	Randomization + Corner Cases: Overflow (big +ve big -ve zero)	-A checker in the testbench to make sure the output is correct -Test Reset Functionality

- Testbench:

```
initial begin
    $display("=====TestBench Start=====");
    error_count = 0;
    correct_count = 0;
    A = 0;
    B = 0;
    assert_reset; //reset

    // Apply all permutations of max pos, max neg, and 0
    // ADDER_2
    A=MAXNEG; B=MAXNEG;
    check_result(-16);
    // ADDER_3
    A=MAXNEG; B=ZERO;
    check_result(MAXNEG);
    // ADDER_8
    A=MAXPOS; B=MAXNEG;
    check_result(-1);
    // ADDER_4
    A=MAXNEG; B=MAXPOS;
    check_result(-1);
    //reset
    assert_reset;
    // ADDER_5
    A=ZERO; B=MAXNEG;
    check_result(MAXNEG);
    // ADDER_6
    A=ZERO; B=ZERO;
    check_result(ZERO);
    // ADDER_7
    A=ZERO; B=MAXPOS;
    check_result(MAXPOS);
    //reset
    assert_reset;
    // ADDER_9
    A=MAXPOS; B=ZERO;
    check_result(MAXPOS);
    // ADDER_10
    A=MAXPOS; B=MAXPOS;
    check_result(14);

    $display("%t: At end of test error count is %0d and correct count = %0d", $time, error_count, correct_count);
    $display("=====TestBench End=====");
    $stop;
end

task check_result (input signed [4:0] expected_result);
    @(negedge clk);
    if (expected_result != C) begin
        error_count = error_count + 1;
        $display("%t: Error: For A= %0d, and B=%0d C should equal %0d but is %0d ",
            $time, A, B, expected_result, C);
    end
    else
        correct_count = correct_count + 1;
endtask

task assert_reset;
    //assert reset
    reset = 1;
    check_result(0);
    //desert reset
    reset = 0;
endtask
```

- Do file:

```
#vlib work
vlog -sv ../../../../VS_code/Session1/Assignment1/Adder/adder.v
../../../../../../VS_code/Session1/Assignment1/Adder/adder_tb.sv +cover -covercells
vsim -voptargs=+acc work.adder_tb -cover
add wave *
coverage save adder_tb.ucdb -onexit
do wave.do
run -all
```

Coverage Report:

```
=====
=== File: ../../../../VS_code/Session1/Assignment1/Adder/adder.v
=====
Statement Coverage:
  Enabled Coverage          Active      Hits      Misses % Covered
  -----
  Stmts                    3          3          0    100.0|

=====Branch Details=====

Branch Coverage for file ../../../../VS_code/Session1/Assignment1/Adder/adder.v --

-----IF Branch-----
   12                      15      Count coming in to IF
   12                      6      if (reset)
   14                      9      else
Branch totals: 2 hits of 2 branches = 100.0%

=====Toggle Details=====

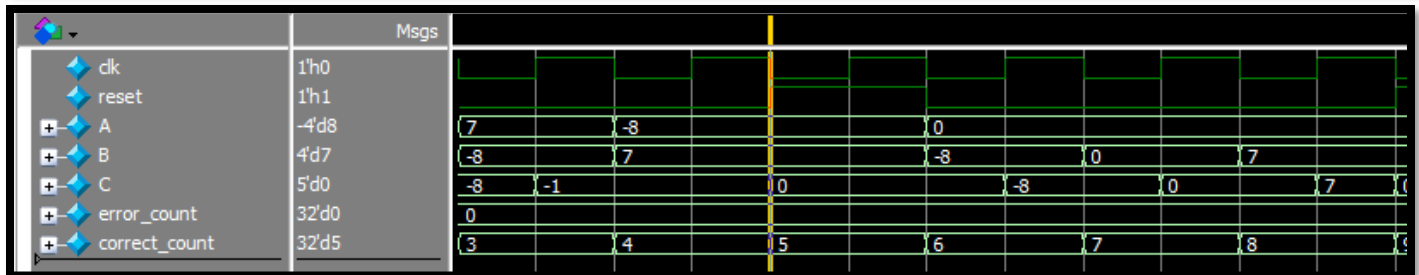
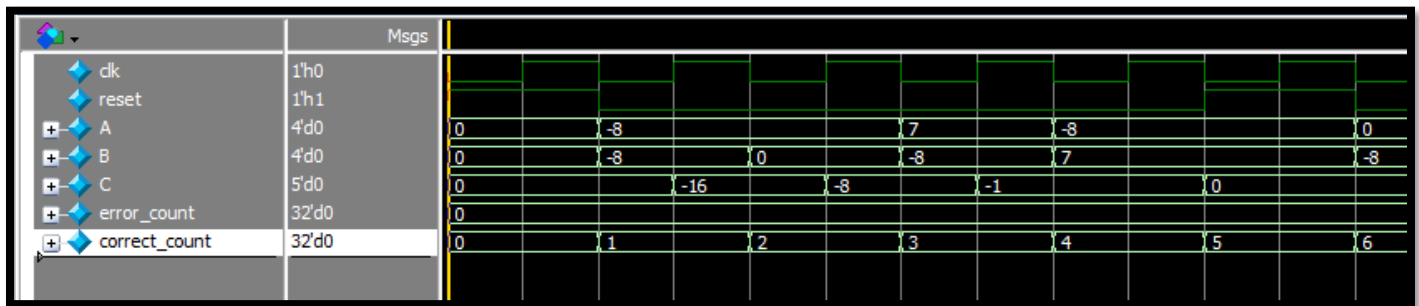
Toggle Coverage for File ../../../../VS_code/Session1/Assignment1/Adder/adder.v --

  Line                      Node      1H->0L      0L->1H  "Coverage"
  -----
    2                      clk          1          1    100.00
    3                      reset         1          1    100.00
    4                      A[3]          1          1    100.00
    4                      A[2]          1          1    100.00
    4                      A[1]          1          1    100.00
    4                      A[0]          1          1    100.00
    5                      B[3]          1          1    100.00
    5                      B[2]          1          1    100.00
    5                      B[1]          1          1    100.00
    5                      B[0]          1          1    100.00
    7                      C[4]          1          1    100.00
    7                      C[3]          1          1    100.00
    7                      C[2]          1          1    100.00
    7                      C[1]          1          1    100.00
    7                      C[0]          1          1    100.00

Total Node Count      =      15
Toggled Node Count    =      15
Untoggled Node Count  =       0

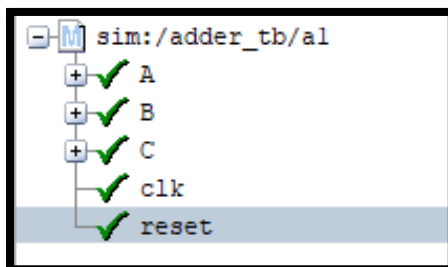
Toggle Coverage       =    100.0% (30 of 30 bins)
```

-Waveforms:

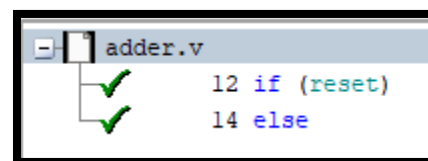


-Coverage Snippets:

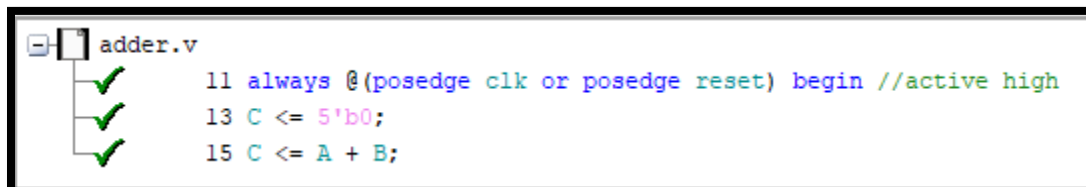
Toggle Coverage



Branch Coverage



Statement Coverage



Design 2: 4-to-2 Priority encoder

-Encoder Design Code:

```
1 module priority_enc (  
2   input  clk,  
3   input  rst,  
4   input  [3:0] D,  
5   output [1:0] Y,    //not reg and assigned in always block  
6   output reg valid  
7 );  
8  
9 always @(posedge clk) begin  
10    if (rst)  
11      Y <= 2'b0;  
12    else  
13      casex (D)  
14        4'b1000: Y <= 0;  
15        4'bX100: Y <= 1;  
16        4'bXX10: Y <= 2;  
17        4'bXXX1: Y <= 3;  
18      endcase    //not all cases covered, need default  
19      valid <= (~|D)? 1'b0: 1'b1;    //inferred latch  
20    end  
21 endmodule
```

```
1 module priority_enc (  
2   input  clk,  
3   input  rst,  
4   input  [3:0] D,  
5   output reg [1:0] Y,    //  
6   output reg valid  
7 );  
8  
9 always @(posedge clk) begin  
10    if (rst) begin  
11      Y <= 2'b0;  
12      valid <= (~|D)? 1'b0: 1'b1; //  
13    end else begin  
14      casex (D)  
15        4'b1000: Y <= 0;  
16        4'bX100: Y <= 1;  
17        4'bXX10: Y <= 2;  
18        4'bXXX1: Y <= 3;  
19        default: Y <= 0; //  
20      endcase  
21      valid <= (~|D)? 1'b0: 1'b1; //  
22    end  
23 end  
24 endmodule
```

-Bugs & fixes:

-Y isn't defined as reg, when its assigned in an always block

-A case where D is all 0's isn't covered in case statement, adding a default statement

-A latch is inferred for value as it's not assigned in the if condition

-missing (begin, end) syntax for if and else

-Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functionality Check
Reset	When the reset is asserted, the output counter value should be low	Directed at the start of the simulation	A checker in the testbench to make sure the output is Zero
i/p Combinations	Verify priority encoding given all i/p combinations	For loop 16 times	A checker in the testbench to make sure the output is correct

- Testbench:

```
27 initial begin
28     integer i;
29
30     $display("=====TestBench Start=====");
31     error_count = 0;
32     correct_count = 0;
33     D = 4'b0000;
34     rst = 0;
35
36     // Test 1: Reset Behavior
37     assert_reset(D);
38
39
40     for (i = 0; i < 16; i = i + 1) begin
41         D = i;
42         check_result(D);
43     end
44
45     D = 4'b0000;
46     check_result(D); //to toggle H-L D[3] for coverage
47
48     assert_reset(D);
49
50
51
52     $display("%t: At end of test error count is %0d and correct count = %0d", $time, error_count, correct_count);
53     $display("=====TestBench End=====");
54     $stop;
55 end

58 task check_result (input [3:0] D);
59
60     @(negedge clk);
61     if (Y !== Y_expected(D) || valid !== valid_expected(D)) begin
62         error_count = error_count + 1;
63         $display("%t: Error: For D= %b, Y should be %0d (got %0d) and valid should be %b (got %b)",
64             $time, D, Y_expected(D), Y, valid_expected(D), valid);
65     end
66     else
67         correct_count = correct_count + 1;
68 endtask
```

```

70 ~ function valid_expected (input [3:0] D_val);
71
72 ~     if(D_val == 4'b0000)
73         return 1'b0;
74 ~     else
75         return 1'b1;
76
77 endfunction

```

```

79 function [1:0] Y_expected (input [3:0] Din); //Golden Model
80
81     if(rst)
82         return 2'b00;
83     else begin
84         if(Din[0] == 1)
85             return 2'b11;
86         else if(Din[1] == 1)
87             return 2'b10;
88         else if(Din[2] == 1)
89             return 2'b01;
90         else if(Din[3] == 1)
91             return 2'b00;
92         else
93             return 2'b00; //as we made the dont care result in design
94     end
95 endfunction
97 task assert_reset(input [3:0] D);
98     @(negedge clk)
99     rst = 1;
100     check_result(D); //Y should equal to 0 regardless of D
101     @(negedge clk)
102     rst = 0;
103 endtask

```

- Do file:

```

#vlib work
vlog -
sv ../../../../VS_code/Session1/Assignment1/Priority_encoder/priority_enc.v
+cover -covercells
vsim -voptargs=+acc work.priority_enc_tb -cover
add wave *
coverage save priority_enc_tb.ucdb -onexit
#do wave.do
run -all

```

Coverage Report:

```
=====  
=== File: ../../../../VS_code/Session1/Assignment1/Priority_encoder/priority_enc.v  
=====
```

Statement Coverage:

Enabled Coverage	Active	Hits	Misses	% Covered
-----	-----	-----	-----	-----
<u>Stmts</u>	9	9	0	100.0

===== Branch Details=====

Branch Coverage for file ../../../../VS_code/Session1/Assignment1/Priority_encoder/priority_enc.v --

-----IF Branch-----

10		19	Count coming in to IF
10	1	2	if (rst) begin
13	1	17	end else begin

Branch totals: 2 hits of 2 branches = 100.0%

===== Toggle Details=====

Toggle Coverage for File ../../../../VS_code/Session1/Assignment1/Priority_encoder/priority_enc.v --

Line	Node	1H->0L	0L->1H	"Coverage"
2	clk	1	1	100.00
3	rst	1	1	100.00
4	D[3]	1	1	100.00
4	D[2]	1	1	100.00
4	D[1]	1	1	100.00
4	D[0]	1	1	100.00
5	Y[1]	1	1	100.00
5	Y[0]	1	1	100.00
6	valid	1	1	100.00

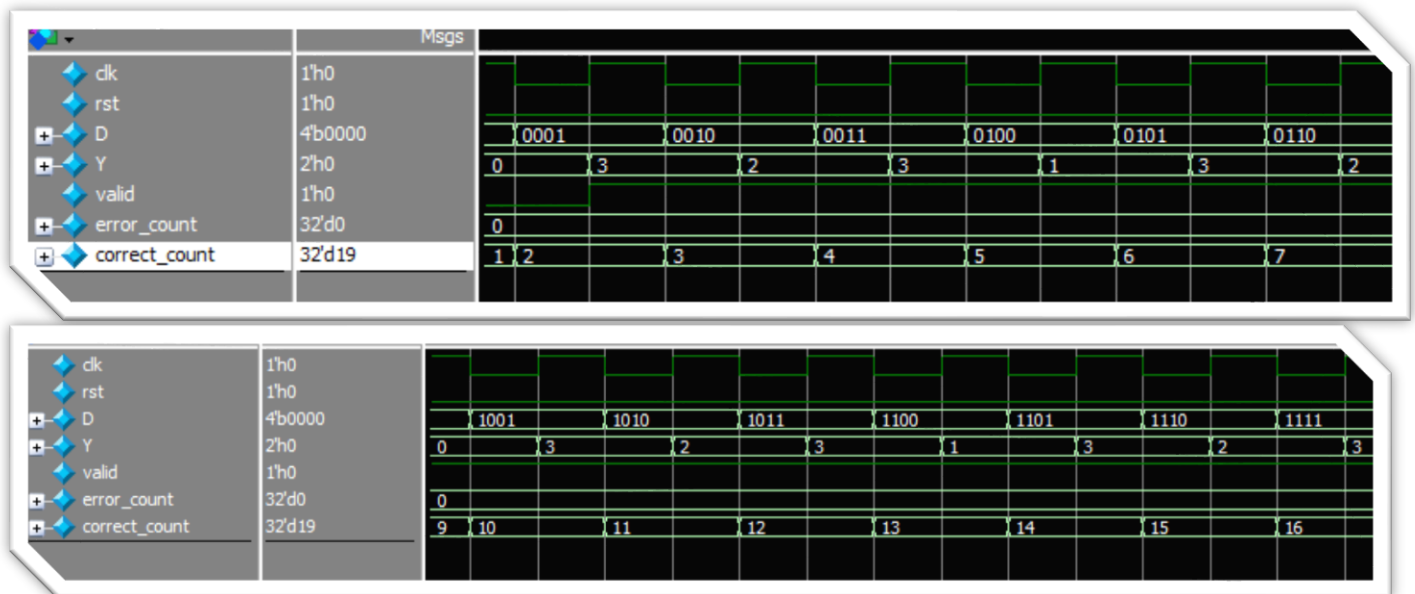
Total Node Count = 9

Toggled Node Count = 9

Untoggled Node Count = 0

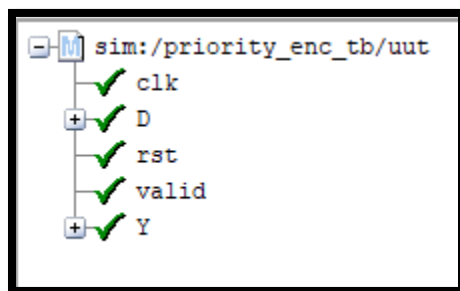
Toggle Coverage = 100.0% (18 of 18 bins)

-Waveforms:

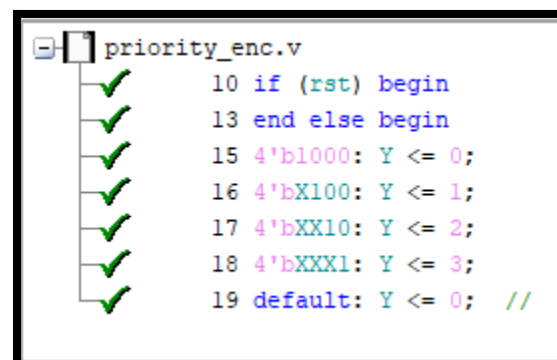


-Coverage Snippets:

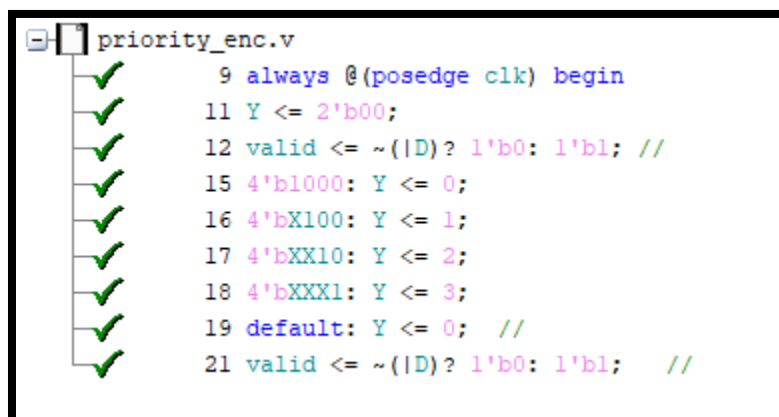
Toggle Coverage



Branch Coverage



Statement Coverage



Design 3: ALU

-ALU Design Code:

```
1 module ALU_4_bit (  
2     input  clk,  
3     input  reset,  
4     input  [1:0] Opcode,    // The opcode  
5     input  signed [3:0] A,  // Input data A in 2's complement  
6     input  signed [3:0] B,  // Input data B in 2's complement  
7  
8     output reg signed [4:0] C // ALU output in 2's complement  
9  
10    );  
11  
12    reg signed [4:0]    Alu_out; // ALU output in 2's complement  
13  
14    localparam          Add          = 2'b00; // A + B  
15    localparam          Sub          = 2'b01; // A - B  
16    localparam          Not_A        = 2'b10; // ~A  
17    localparam          ReductionOR_B = 2'b11; // |B  
18  
19    // Do the operation  
20    always @* begin  
21        case (Opcode)  
22            Add:          Alu_out = A + B;  
23            Sub:          Alu_out = A - B;  
24            Not_A:        Alu_out = ~A;  
25            ReductionOR_B: Alu_out = |B;  
26            default:      Alu_out = 5'b0;  
27        endcase  
28    end // always @ *  
29  
30    // Register output C  
31    always @(posedge clk or posedge reset) begin  
32        if (reset)  
33            C <= 5'b0;  
34        else  
35            C <= Alu_out;  
36    end  
37  
38    endmodule
```

-Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functionality Check
ALU_1	Reset behavior: Output C must go to 0 regardless of opcode or inputs.	Directed: Assert reset at start and during random operation.	Checker: if (C != 0) error.
ALU_2	Arithmetic Boundary: Signed addition and subtraction hitting overflow/extreme values.	Directed: Max positive (7), Max negative (-8), and Zero.	Checker: Compare C against expected signed result.
ALU_3	Bitwise Logic: Correct inversion of A and Reduction OR of B.	Directed: Patterns like 4'b1010, 4'b0000, 4'b1111.	Checker: if (C != ~A)
ALU_4	Opcode Transitions: Ensure output updates correctly when switching opcodes for fixed A/B.	Directed: Fixed A and B, loop through all 4 opcodes.	Checker: Match result per opcode cycle.

- Testbench:

```
0 task compute_expected;
1
2 if (Opcode==Add) expected_C = A + B;
3 else if (Opcode==Sub) expected_C = A - B;
4 else if (Opcode==Not_A) expected_C = ~A;
5 else if (Opcode==ReductionOR_B) expected_C = |B;
6 else expected_C = 5'b00;

    check_result;
endtask

task assert_check;
    reset = 1;
    @(negedge clk);
    if(C!=0)begin
        error_count=error_count+1;
        $display("error");
    end
    else
        correct_count=correct_count+1;
    reset=0;
endtask

task check_result;
    @(negedge clk);
    if(C==expected_C)begin
        error_count=error_count+1;
        $display("%t: Error: For A=%0d ,B=%0d ,C should equal %0d but is %0d",
            $time, A,B, expected_C, C);
    end
    else
        correct_count=correct_count+1;
endtask
```

```

initial begin
    A=0;
    B=0;
    error_count=0;
    correct_count=0;

// Test1
assert_check;

//Test2 ADD
    A=MAXNEG;B=MAXNEG;Opcode=Add;
    compute_expected;
    A=MAXPOS;B=MAXPOS;
    compute_expected;
    A=MAXPOS;B=MAXNEG;
    compute_expected;
    A=MAXNEG;B=MAXPOS;
    compute_expected;
    A=0;B=MAXPOS;
    compute_expected;
    A=MAXNEG;B=0;
    compute_expected;
    A=0;B=MAXNEG;
    compute_expected;
    A=MAXPOS;B=0;
    compute_expected;
    A=0;B=0;
    compute_expected;
//Test3 sub
    A=MAXNEG;B=MAXNEG;Opcode=Sub;
    compute_expected;
    A=MAXPOS;B=MAXPOS;
    compute_expected;

```

```

    A=MAXNEG;B=MAXPOS;
    compute_expected;
    A=0;B=MAXPOS;
    compute_expected;
    A=MAXNEG;B=0;
    compute_expected;
    A=0;B=MAXNEG;
    compute_expected;
    A=MAXPOS;B=0;
    compute_expected;
    A=0;B=0;
    compute_expected;

```

```

//Test4 ~A
repeat(10)begin
    Opcode=Not_A;
    A=$random;
    compute_expected;
end

```

```

//Test4 |B
Opcode=ReductionOR_B;
repeat(10)begin
    B=$random;
    compute_expected;

```

```

//Test4 |B
Opcode=ReductionOR_B;
repeat(10)begin
    B=$random;
    compute_expected;
end

```

```

//Test 5

```

```

A = 3;
B = -2;

```

```

Opcode = 2'b00; compute_expected;
Opcode = 2'b01; compute_expected;
Opcode = 2'b10; compute_expected;
Opcode = 2'b11; compute_expected;

```

```

// Test 6
assert_check;
    $display("%t: At end of test error count is %0d", $stop);

```

```

end

```

```

endmodule

```

- Do file

```
1  vlib work
2  vlog ALU.v ALU_tb.sv +cover -covercells
3  vsim -voptargs="+acc" work.ALU_tb -coverage
4  add wave *
5  coverage save ALU_tb.ucdb -onexit
6  run -all
```

```
=====
=== Instance: /ALU_tb/dut
=== Design Unit: work.ALU_4_bit
=====
```

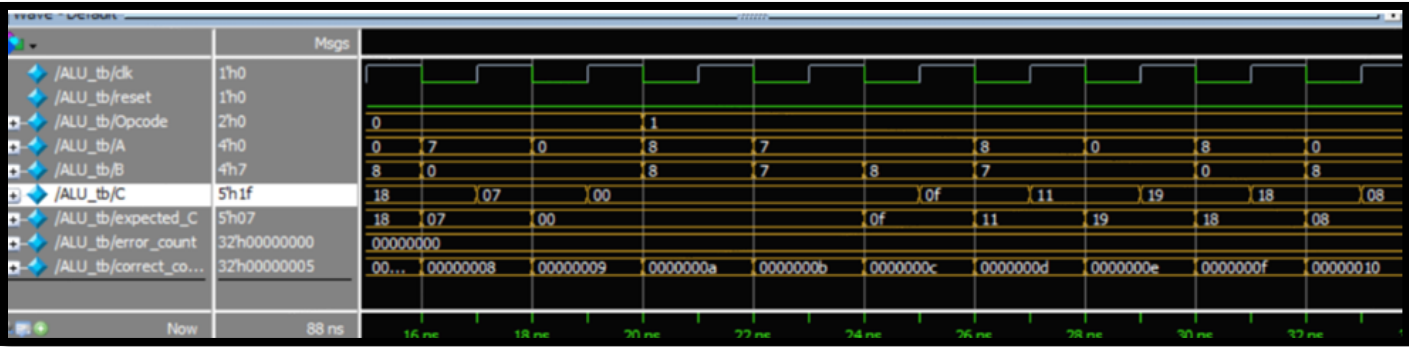
Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	9	9	0	100.00%

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	44	44	0	100.00%

-Waveforms:



Design 4: DSP

-DSP Design Code:

```
1  module DSP(A, B, C, D, clk, rst_n, P);
2  parameter OPERATION = "ADD";
3  input  [17:0] A, B, D;
4  input  [47:0] C;
5  input clk, rst_n;
6  output reg [47:0] P;
7
8  reg [17:0] A_reg_stg1, A_reg_stg2, B_reg, D_reg,
9  adder_out_stg1, adder_out_stg2;
10 reg [47:0] C_reg, mult_out;
11
12 always @(posedge clk or negedge rst_n) begin
13     if (!rst_n) begin
14         // reset
15         A_reg_stg1 <= 0;
16         A_reg_stg2 <= 0;
17         B_reg <= 0; //missing C_reg initialization
18         D_reg <= 0;
19         adder_out_stg1 <= 0;
20         mult_out <= 0;
21         P <= 0;
22     end
23     else begin
24         A_reg_stg1 <= A;
25         A_reg_stg2 <= A_reg_stg1;
26         B_reg <= B;
27         C_reg <= C;
28         D_reg <= D;
29         adder_out_stg2 <= adder_out_stg1; //adder_out_stg1 should have been updated first
30         if (OPERATION == "ADD") begin
31             adder_out_stg1 <= D_reg + B_reg;
32             P <= mult_out + C_reg;
33         end
34         else if (OPERATION == "SUBTRACT") begin
35             adder_out_stg1 <= D_reg - B_reg;
36             P <= mult_out - C_reg; //mult_out should have been updated first
37         end //missing else default
38         mult_out <= A_reg_stg2 * adder_out_stg2;
39     end
40 end
41
42 endmodule
```

-Bugs & fixes:

-Y

-Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functionality Check
Reset	When the reset is asserted, the output counter value should be low	Directed at the start of the simulation	A checker in the testbench to make sure the output is Zero
i/p Combinations	Verify priority encoding given all i/p combinations	For loop 16 times	A checker in the testbench to make sure the output is correct

- Testbench:

```
27 initial begin
28     integer i;
29
30     $display("=====TestBench Start=====");
31     error_count = 0;
32     correct_count = 0;
33     D = 4'b0000;
34     rst = 0;
35
36     // Test 1: Reset Behavior
37     assert_reset(D);
38
39
40     for (i = 0; i < 16; i = i + 1) begin
41         D = i;
42         check_result(D);
43     end
44
45     D = 4'b0000;
46     check_result(D); //to toggle H-L D[3] for coverage
47
48     assert_reset(D);
49
50
51
52     $display("%t: At end of test error count is %0d and correct count = %0d", $time, error_count, correct_count);
53     $display("=====TestBench End=====");
54     $stop;
55 end

58 task check_result (input [3:0] D);
59
60     @(negedge clk);
61     if (Y !== Y_expected(D) || valid !== valid_expected(D)) begin
62         error_count = error_count + 1;
63         $display("%t: Error: For D= %b, Y should be %0d (got %0d) and valid should be %b (got %b)",
64             $time, D, Y_expected(D), Y, valid_expected(D), valid);
65     end
66     else
67         correct_count = correct_count + 1;
68 endtask
```

```

70 ~ function valid_expected (input [3:0] D_val);
71
72 ~     if(D_val == 4'b0000)
73         return 1'b0;
74 ~     else
75         return 1'b1;
76
77 endfunction

```

```

79 function [1:0] Y_expected (input [3:0] Din); //Golden Model
80
81     if(rst)
82         return 2'b00;
83     else begin
84         if(Din[0] == 1)
85             return 2'b11;
86         else if(Din[1] == 1)
87             return 2'b10;
88         else if(Din[2] == 1)
89             return 2'b01;
90         else if(Din[3] == 1)
91             return 2'b00;
92         else
93             return 2'b00; //as we made the dont care result in design
94     end
95 endfunction
97 task assert_reset(input [3:0] D);
98     @(negedge clk)
99     rst = 1;
100     check_result(D); //Y should equal to 0 regardless of D
101     @(negedge clk)
102     rst = 0;
103 endtask

```

- Do file:

```

#vlib work
vlog -
sv ../../../../VS_code/Session1/Assignment1/Priority_encoder/priority_enc.v
+cover -covercells
vsim -voptargs=+acc work.priority_enc_tb -cover
add wave *
coverage save priority_enc_tb.ucdb -onexit
#do wave.do
run -all

```

Coverage Report:

```
=====  
=== File: ../../../../VS_code/Session1/Assignment1/Priority_encoder/priority_enc.v  
=====
```

Statement Coverage:

Enabled Coverage	Active	Hits	Misses	% Covered
-----	-----	-----	-----	-----
<u>Stmts</u>	9	9	0	100.0

===== Branch Details=====

Branch Coverage for file ../../../../VS_code/Session1/Assignment1/Priority_encoder/priority_enc.v --

-----IF Branch-----

10		19	Count coming in to IF
10	1	2	if (rst) begin
13	1	17	end else begin

Branch totals: 2 hits of 2 branches = 100.0%

===== Toggle Details=====

Toggle Coverage for File ../../../../VS_code/Session1/Assignment1/Priority_encoder/priority_enc.v --

Line	Node	1H->0L	0L->1H	"Coverage"
2	clk	1	1	100.00
3	rst	1	1	100.00
4	D[3]	1	1	100.00
4	D[2]	1	1	100.00
4	D[1]	1	1	100.00
4	D[0]	1	1	100.00
5	Y[1]	1	1	100.00
5	Y[0]	1	1	100.00
6	valid	1	1	100.00

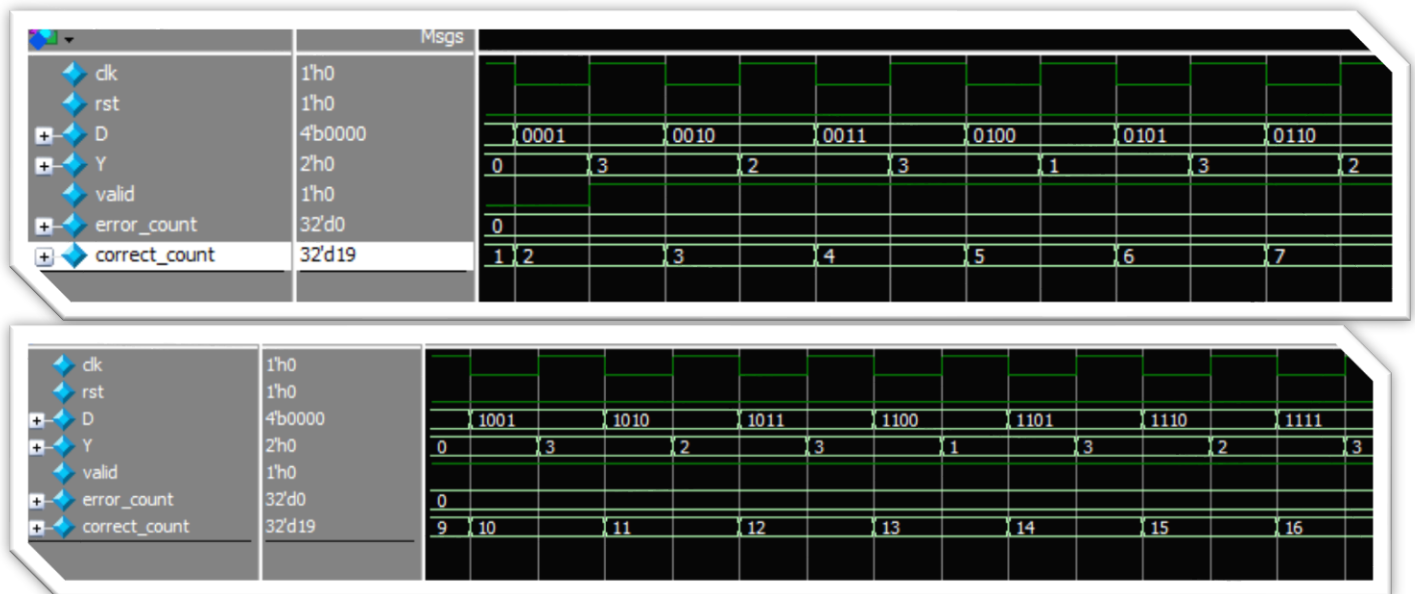
Total Node Count = 9

Toggled Node Count = 9

Untoggled Node Count = 0

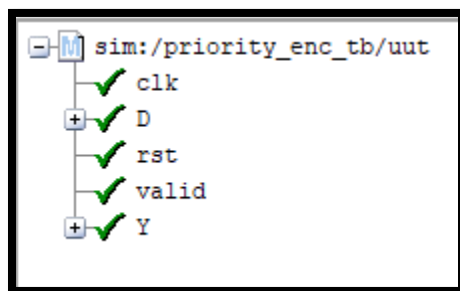
Toggle Coverage = 100.0% (18 of 18 bins)

-Waveforms:

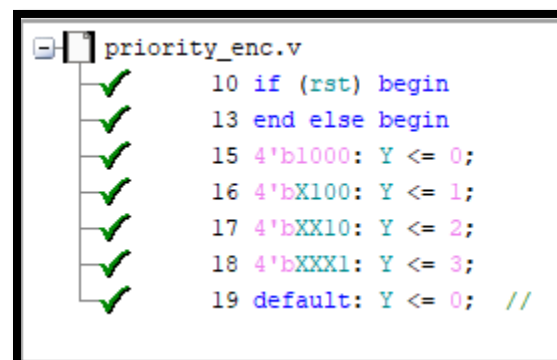


-Coverage Snippets:

Toggle Coverage



Branch Coverage



Statement Coverage

